gradual typing

kei rockwell

motivation

- Dynamically typed languages are popular due to ease of rapid development and prototyping
- Statically typed languages offer compile-type error detection, better maintainability once program structure settles, compiler optimizations
- In practice, it is desirable to add typing information to a dynamic program at a later date to benefit from static type checking
- Want a type theory that enables convenient evolution of code between dynamic and static settings
- Still should catch static type errors when given information is inconsistent

`(num -> num)(num * ?).fst` **OK** `(num -> num)(bool * ?).fst` **BAD**

Siek and Taha (2006)

Gradual Typing for Functional Languages

- Consistency relation: types are consistent if they are the same or if at least one is ?
  - Care is needed for arrow types
- Fully annotated programs should share semantics with static language
- Programs with no annotations should share semantics with dynamic language
- Define a Gradually Typed Lambda Calculus (GTLC) satisfying these
  - Prove type safety of GTLC
  - Show it catches motivating errors
- **Future Work:**
  - Investigate richer type systems (e.g. lists, algebraic data types, etc.)
  - Implement into mainstream language

Siek, Vitousek, Cimini, and Boyland (2015)

Refined Criteria For Gradual Typing

- Identified a “dilution” of the term “gradual typing” to mean simply allowing for static and dynamic types
- Partially attribute this to a lack of existing formalization for ensuring “convenient evolution” from untyped to typed code
- What should a “gradual guarantee” look like?

Siek, Vitousek, Cimini, and Boyland (2015)

a motivating program fragment

```
fn gcd(a, b) {
  if a == 0 then b
  else gcd(b, a % b)
}
```

```
fn gcd(a: int, b) {
  if a == 0 then b
  else gcd(b, a % b)
}
```

```
fn gcd(a: int, b: int) -> bool {
  if a == 0 then b
  else gcd(b, a % b)
}
```

```
fn gcd(a: int, b: int) -> int {
  if a == 0 then b
  else gcd(b, a % b)
}
```

Siek, Vitousek, Cimini, and Boyland (2015)

another motivating fragment

"Incorrect" annotations can also trigger runtime errors:

```
fn gcd(a: int, b) {
  if a == 0 then b
  else gcd(b, a % b)
}
```

```
fn lcm_a(a: int, b) {
  a * b / gcd(a, b)
}
fn lcm_b(a: int, b: float) {
  a * b / gcd(a, b)
}
lcm_a(6, 3) // Okay
lcm_b(6, 3) // Runtime Error
```

Siek, Vitousek, Cimini, and Boyland (2015)

the gradual guarantee

- If a gradually typed program is well typed, removing annotations produces a well typed program
- If a program evaluates to a value, removing annotations produces a program that evaluates to the same value
- Adding annotations that **agree with some fully annotated, well typed** version of a program produces a well typed program

Siek, Vitousek, Cimini, and Boyland (2015)

formalizing the gradual guarantee

Define a **partial ordering on types and terms**. Read T ⊑ T' as "**T is more precise than T'**"

Type Precision

$$\boxed{T \sqsubseteq T}$$

$$\frac{}{T \sqsubseteq \star} \qquad \frac{}{B \sqsubseteq B} \qquad \frac{T_1 \sqsubseteq T_3 \quad T_2 \sqsubseteq T_4}{T_1 \to T_2 \sqsubseteq T_3 \to T_4}$$

Term Precision for the GTLC

$$\boxed{e \sqsubseteq e}$$

$$\frac{}{k \sqsubseteq k} \qquad \frac{}{x \sqsubseteq x} \qquad \frac{T_1 \sqsubseteq T_2 \quad e_1 \sqsubseteq e_2}{\lambda x{:}T_1.\, e_1 \sqsubseteq \lambda x{:}T_2.\, e_2} \qquad \frac{e_1 \sqsubseteq e_2 \quad e_1' \sqsubseteq e_2'}{(e_1\ e_1')^{\ell} \sqsubseteq (e_2\ e_2')^{\ell}}$$

Siek, Vitousek, Cimini, and Boyland (2015)

formalizing the gradual guarantee

Suppose $e \sqsubseteq e'$ and $\vdash e : T$. Then:

1. $\vdash e' : T'$ for some T' such that $T \sqsubseteq T'$
2. If $e \Downarrow v$, then $e' \Downarrow v'$ for some v' such that $v \sqsubseteq v'$
   - If $e \Uparrow$, then $e' \Uparrow$ (where $\Uparrow$ denotes a failure to evaluate to a value)
3. If $e' \Downarrow v'$, then either $e \Downarrow v$ for some $v \sqsubseteq v'$ or a runtime type error occurs
   - If $e' \Uparrow$, then either $e \Uparrow$ or a runtime type error occurs

Siek, Vitousek, Cimini, and Boyland (2015)

evaluating the gradual guarantee

- Proved the GTLC satisfies the gradual guarantee
- Examined several practical languages designed to be gradual
  - **Generally, these failed to satisfy the guarantee**
  - Proving the gradual guarantee for more complex languages is challenging
  - Potential barrier to efficient implementation
- **Future work**
  - More consideration of painless evolution is needed for gradual language design
  - Proving the gradual guarantee for a “full blown” language
  - **Efficient implementation of languages satisfying this**

Siek (2009)

Space-Efficient Blame Tracking for Gradual Types

- Identifies two problems with casts inserted when compiling gradually typed code:
  - Finding the cause of run-time type errors is challenging (“blame tracking”)
  - Casts can become very expensive in terms of space
- Existing solutions to each problem individually
- Introduce **high fidelity blame tracking**
- Give space efficient blame semantics

```
line 3: cast error!
expected value of type: (int → int) → int
but, for the indicated location,
   got a value of type: bool
the producer is to blame: g
location that triggered the error: line 2
```

Siek (2009)

Space-Efficient Blame Tracking for Gradual Types

- Mechanized proof of type safety
- Proof of blame safety
- Prove that the space-efficient semantics are equivalent to inefficient version
- **Future work**:
  - Still entirely within the GTLC
  - Space is exponential in the largest type of a program

Takikawa, Feltey, Greenman, New, Vitek, and Felleisen (2016)

Is Sound Gradual Typing Dead?

- Identified no benchmarking or performance challenges in existing practice
- Made benchmarking framework for Typed Racket testing varying precision levels
- Found partially typed code resulted in extreme performance degradation
  - **> 80% of execution time in all partially typed benchmarks was runtime checks**
  - Generally over 3x slowdown
- **Future Work:**
  - Apply to other languages
  - Check more sophisticated JIT/optimization techniques
  - Examine potential of programmer practice minimizing efficiency loss

Bauman, Bolz-Teriek, Siek, and Tobin-Hochstadt (2017)

Sound Typing: Only Mostly Dead

- Previous profiling work identified potential speedups by more sophisticated JIT
- Use Pycket: racket implementation with tracing JIT and optimizer based on PyPy
- Improved the representation of dynamic types for this implementation
- Evaluate against the benchmarks developed in prior work
  - **Significant reduction in gradual typing overhead**
  - Overhead compared to fully untyped, JIT code
- Future Work
  - Can some techniques be extended to non-tracing JIT
  - Typed Racket's design requires entire modules to be typed or untyped
  - **Still room for improvement** — cost is still too high on some benchmarks

Muehlboeck and Tate (2017)

Sound gradual typing is nominally alive and well

- Observe that existing implementations of gradual typing designed for production attempt to add gradual types to existing implementations
- New properties for gradual type system to facilitate efficient performance
- Evaluate prototype compiler for this and C#
- **Future Work:**
  - Support structural types
  - Support first class functions

Conclusion

- Efficient implementation of gradual types remains a key challenge
  - While there have been improvements, performance is still a barrier to adaptation
  - When building on top of existing languages, interoperability with untyped libraries is of particular concern
- Implementations of gradual typing tends to be limited to simple type systems
  - Research is actively occurring on more complex types systems (e.g. Dependent Types)
  - However, these extensions are still generally missing important theoretical properties

questions?